

Backend Overview — b-secure

Django Backend

Version: 1.0.0 **Framework:** Django 5.x + Django REST Framework 3.15+
Runtime: Python 3.12+ **Protocol:** HTTP (REST) + WebSocket (Django Channels / ASGI)

Table of Contents

- 1. [Technology Decisions](#)
 - 2. [Project Folder Structure](#)
 - 3. [Django App Breakdown](#)
 - 4. [Local Development Setup](#)
 - 5. [Environment Variables](#)
 - 6. [Settings Architecture](#)
 - 7. [URL Routing](#)
 - 8. [ASGI vs WSGI](#)
 - 9. [Code Style & Conventions](#)
 - 10. [Common Utilities](#)
-

1. Technology Decisions

Why Django?

Concern	Decision	Reason
Web framework	Django 5.x	Batteries-included: ORM, admin, auth, migrations. Fast to build production-grade apps.
API layer	Django REST Framework	Industry standard, excellent serializer system, viewsets, permissions.

Concern	Decision	Reason
Real-time	Django Channels 4.x	First-class WebSocket support on ASGI without leaving Django ecosystem.
Task queue	Celery 5.x	Mature, battle-tested. Native Django integration. Redis as broker.
Database	PostgreSQL 16 + PostGIS	ACID compliance, geospatial queries (guard proximity), JSON fields, full-text search.
Cache	Redis 7.x	Used for: Django cache, Celery broker, Channels layer, session store, rate limiting.
Auth	JWT (SimpleJWT)	Stateless, scales horizontally. Short-lived access tokens + rotating refresh tokens.

Why not FastAPI / Node?

- Django's ORM + migrations significantly reduce development time for a data-heavy platform like b-secure.
- Django Channels handles WebSocket natively — no need for a separate Node.js service.
- Django Admin gives a free internal management interface during early development.
- Strong ecosystem for payments, auth, storage, and background tasks.

2. Project Folder Structure

```

bsecure_backend/
├── manage.py           # Django management entry
point
├── celery_app.py       # Celery application
instance
└── pytest.ini          # Pytest configuration

```

— .env.example	# Example environment
variables	
— .env	# Local environment (never
commit)	
— Dockerfile	# Production Docker image
— docker-compose.yml	# Local dev multi-service
setup	
— .github/	
— workflows/	
— ci.yml	# Run tests on every PR
— deploy.yml	# Deploy on merge to main
— requirements/	
— base.txt	# Shared dependencies
— development.txt	# Dev-only (debug toolbar,
factory boy)	
— production.txt	# Prod-only (gunicorn,
sentry, etc.)	
— config/	# Project-level
configuration	
— __init__.py	
— settings/	
— __init__.py	# All shared settings
— base.py	# Local overrides
— development.py	# Production overrides
— production.py	
— urls.py	# Root URL configuration
— asgi.py	# ASGI app (HTTP +
WebSocket routing)	
— wsgi.py	# WSGI app (fallback /
admin-only)	
— apps/	# All Django application
modules	
— authentication/	# OTP, JWT, social auth
— models.py	# OTPToken,
BlacklistedToken	
— serializers.py	
— views.py	
— urls.py	
— throttles.py	# Custom OTP rate limiter
— tests/	
— users/	# User profiles, addresses,
contacts	
— models.py	# UserProfile, Address,
EmergencyContact	
— serializers.py	
— views.py	
— urls.py	
— signals.py	# Post-save hooks (e.g.
create wallet)	

			└ tests/	
			└ guards/	# Guard profiles,
			documents, availability	
			└ models.py	# GuardProfile,
			GuardDocument, GuardAvailability	
			└ serializers.py	
			└ views.py	
			└ urls.py	
			└ services.py	# Guard matching logic
			└ signals.py	
			└ tests/	
			└ bookings/	# Full booking lifecycle
			└ models.py	# Booking,
			BookingBroadcast, Session	
			└ serializers.py	
			└ views.py	
			└ urls.py	
			└ services.py	# Booking state machine
			logic	
			└ tasks.py	# Celery tasks (broadcast,
			expire, checkin)	
			└ tests/	
			└ tracking/	# Real-time location
			└ models.py	# LocationSnapshot
			└ consumers.py	# Django Channels WebSocket
			consumers	
			└ routing.py	# WebSocket URL routing
			└ serializers.py	
			└ views.py	# Session history API
			└ urls.py	
			└ tests/	
			└ payments/	# Wallet, transactions,
			payouts	
			└ models.py	# Wallet, Transaction,
			PaymentOrder, Payout	
			└ serializers.py	
			└ views.py	
			└ urls.py	
			└ services.py	# Payment processing logic
			└ tasks.py	# Celery: process payment,
			payout, invoice	
			└ webhooks.py	# Razorpay / Stripe webhook
			handlers	
			└ tests/	
			└ notifications/	# Push, SMS, Email
			└ models.py	# NotificationLog,
			NotificationPreference	
			└ serializers.py	
			└ views.py	

			urls.py	
			services/	
			push.py	# FCM push notification
	sender			
			sms.py	# Twilio SMS sender
			email.py	# SendGrid email sender
			tasks.py	# Async notification
	dispatch			
			tests/	
			sos/	# SOS alerts, incidents
			models.py	# SOSAlert, Incident,
	IncidentEvidence			
			serializers.py	
			views.py	
			urls.py	
			consumers.py	# WebSocket consumer for
	admin SOS feed			
			routing.py	
			services.py	# SOS trigger orchestration
			tasks.py	# Escalation timers,
	emergency contact alerts			
			tests/	
			reviews/	# Ratings and reviews
			models.py	# Review
			serializers.py	
			views.py	
			urls.py	
			tests/	
			admin_panel/	# Admin-specific APIs
			serializers.py	# Admin-level serializers
	(more fields)			
			views/	
			dashboard.py	
			users.py	
			guards.py	
			bookings.py	
			sos.py	
			payments.py	
			analytics.py	
			urls.py	
			consumers.py	# Admin live dashboard
	WebSocket			
			routing.py	
			tests/	
			analytics/	# Aggregated metrics
			models.py	# DailyStats (materialized/
	denormalized)			
			views.py	
			urls.py	
			tasks.py	# Nightly aggregation tasks

```

├── tests/
├── utils/                    # Shared utilities (no
Django models)
│   ├── __init__.py
│   └── permissions.py       # Custom DRF permission
classes
│   └── pagination.py        # Standard pagination
config
│   └── validators.py        # Phone, geo, file
validators
│   ├── exceptions.py        # Custom exception handler
│   └── helpers.py           # OTP generation, geo
distance, etc.
│   ├── mixins.py            # Common ViewSet mixins
│   └── storage.py            # S3 pre-signed URL helper

```

3. Django App Breakdown

Each app follows a consistent internal pattern:

```

app_name/
├── models.py                → Database models (single source of truth)
├── serializers.py           → DRF serializers (validation +
representation)
├── views.py                 → DRF ViewSets / APIViews (HTTP handlers)
├── urls.py                  → URL patterns for this app
├── services.py              → Business logic (NOT in views, NOT in
models)
├── tasks.py                 → Celery async tasks
├── signals.py               → Django signals (side effects)
├── consumers.py             → Django Channels WebSocket consumers (if
real-time)
├── routing.py               → WebSocket URL routing (if real-time)
├── admin.py                 → Django admin registration
├── tests/
│   ├── test_models.py
│   ├── test_views.py
│   ├── test_services.py
│   └── test_tasks.py

```

App Responsibilities

App	Owns	Does NOT own
authentication	OTP tokens, JWT issuance, social auth	User profile data
users	UserProfile, Address, EmergencyContact	Auth tokens

App	Owns	Does NOT own
guards	GuardProfile, Documents, Availability	Booking logic
bookings	Booking lifecycle, Session OTP	Payment processing
tracking	LocationSnapshot, WS consumers	Booking state
payments	Wallet, Transactions, Payouts	Booking creation
notifications	Dispatch logic, NotificationLog	Business triggers
sos	SOSAlert, Incident	Notification dispatch
reviews	Review model	Booking lifecycle
admin_panel	Admin API views	Core business models
analytics	Aggregated stats models	Raw data writes

The Service Layer Pattern

Business logic lives in `services.py`, not in views or models:

```
# BAD – logic in view
class BookingViewSet(viewsets.ModelViewSet):
    def create(self, request):
        # 50 lines of business logic here – WRONG
        ...

# GOOD – view delegates to service
class BookingViewSet(viewsets.ModelViewSet):
    def create(self, request):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        booking = BookingService.create_booking(
            user=request.user,
            data=serializer.validated_data
        )
        return Response(BookingSerializer(booking).data,
            status=201)

# services.py
class BookingService:
    @staticmethod
    def create_booking(user, data):
        # All business logic here: validation, guard matching,
```

```
# price calculation, notification triggers, etc.
```

```
...
```

4. Local Development Setup

Prerequisites

- Python 3.12+
- Docker & Docker Compose
- Git

Step 1: Clone and set up Python environment

```
git clone https://github.com/your-org/bsecure-backend.git
cd bsecure-backend
```

```
# Create virtual environment
```

```
python3.12 -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

```
# Install dependencies
```

```
pip install -r requirements/development.txt
```

Step 2: Start infrastructure services (PostgreSQL + Redis)

```
# Start only DB and Redis (not the app yet)
```

```
docker-compose up -d db redis
```

Step 3: Configure environment

```
cp .env.example .env
```

```
# Edit .env with your local values (see Environment Variables
section)
```

Step 4: Run migrations and seed data

```
# Run all migrations
```

```
python manage.py migrate
```

```
# Load PostGIS extension (first time only)
```

```
python manage.py shell -c "
```

```
from django.db import connection
```

```
with connection.cursor() as c:
```

```
    c.execute('CREATE EXTENSION IF NOT EXISTS postgis;')
```

```
"
```

```
# Then re-run migrate
```



```
python manage.py migrate
```

```
# Create superuser for Django admin
```

```
python manage.py createsuperuser
```

```
# (Optional) Load development seed data
```

```
python manage.py loaddata fixtures/dev_seed.json
```

Step 5: Run the development server

```
# Run ASGI server (supports both HTTP and WebSocket)
```

```
daphne -b 0.0.0.0 -p 8000 config.asgi:application
```

```
# In a separate terminal: run Celery worker
```

```
celery -A celery_app worker -l info -Q  
high_priority,default,low_priority --concurrency=4
```

```
# In another terminal: run Celery beat scheduler
```

```
celery -A celery_app beat -l info --scheduler  
django_celery_beat.schedulers:DatabaseScheduler
```

Step 6: Verify the setup

```
# Health check
```

```
curl http://localhost:8000/api/health/
```

```
# Expected response:
```

```
# {"status": "ok", "db": "connected", "redis": "connected",  
  "version": "1.0.0"}
```

Using Docker Compose (Full Stack)

```
# Start everything (API + Celery + DB + Redis)
```

```
docker-compose up --build
```

```
# Run migrations inside container
```

```
docker-compose exec api python manage.py migrate
```

```
# Create superuser inside container
```

```
docker-compose exec api python manage.py createsuperuser
```

5. Environment Variables

All secrets and environment-specific config live in `.env`. Never hardcode values. Use `django-environ` to load them.

```
# .env.example
```

Django Core

DJANGO_SETTINGS_MODULE=config.settings.development
SECRET_KEY=your-very-long-random-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1,0.0.0.0

Database (PostgreSQL + PostGIS)

DB_NAME=bsecure
DB_USER=bsecure
DB_PASSWORD=dev_password
DB_HOST=localhost
DB_PORT=5432

Redis

REDIS_URL=redis://localhost:6379/0

AWS S3

AWS_ACCESS_KEY_ID=your-access-key
AWS_SECRET_ACCESS_KEY=your-secret-key
AWS_S3_BUCKET_NAME=bsecure-media-dev
AWS_S3_REGION_NAME=ap-south-1

JWT

JWT_ACCESS_TOKEN_LIFETIME_MINUTES=15
JWT_REFRESH_TOKEN_LIFETIME_DAYS=30

Firebase (Push Notifications)

FIREBASE_SERVICE_ACCOUNT_KEY_PATH=/path/to/serviceAccountKey.json

Twilio (SMS)

TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your-auth-token
TWILIO_PHONE_NUMBER=+1234567890

Razorpay (Payments)

RAZORPAY_KEY_ID=rzp_test_XXXXXXX
RAZORPAY_KEY_SECRET=your-razorpay-secret
RAZORPAY_WEBHOOK_SECRET=your-webhook-secret

Stripe (Payments - International)

STRIPE_SECRET_KEY=sk_test_XXXXXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXX
STRIPE_PUBLISHABLE_KEY=pk_test_XXXXXXX

SendGrid (Email)

SENDGRID_API_KEY=SG.XXXXXXXXXXXXXXX
DEFAULT_FROM_EMAIL=noreply@bsecure.in

Google Maps

GOOGLE_MAPS_API_KEY=AIzaXXXXXXXXXXXXXXXXXX

Sentry (Error Tracking)

SENTRY_DSN=https://xxxxx@sentry.io/xxxxx

```
# CORS (comma-separated list)
CORS_ALLOWED_ORIGINS=http://localhost:3000,http://localhost:19006

# MSG91 (SMS - Alternative to Twilio)
MSG91_AUTH_KEY=your-msg91-key
MSG91_TEMPLATE_ID=your-otp-template-id

# IDfy (Background Verification)
IDFY_API_KEY=your-idfy-api-key
IDFY_ACCOUNT_ID=your-idfy-account-id

# Admin Panel
ADMIN_PANEL_URL=http://localhost:3000
FRONTEND_URL=http://localhost:19006
```

6. Settings Architecture

Settings are split into base + environment-specific files, loaded via `DJANGO_SETTINGS_MODULE`.

```
# config/settings/base.py (excerpt – key sections)

import environ
from pathlib import Path

env = environ.Env()
BASE_DIR = Path(__file__).resolve().parent.parent

# Read .env file
environ.Env.read_env(BASE_DIR / '.env')

SECRET_KEY = env('SECRET_KEY')

INSTALLED_APPS = [
    # Django built-ins
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'django.contrib.gis',          # PostGIS support

    # Third-party
    'rest_framework',
    'rest_framework_simplejwt',
    'rest_framework_simplejwt.token_blacklist',
    'corsheaders',
    'channels',
```

```

'django_filters',
'django_celery_beat',
'django_celery_results',
'stores',
'social_django',
'drf_spectacular',          # OpenAPI schema generation

# b-secure apps
'apps.authentication',
'apps.users',
'apps.guards',
'apps.bookings',
'apps.tracking',
'apps.payments',
'apps.notifications',
'apps.sos',
'apps.reviews',
'apps.admin_panel',
'apps.analytics',
]

AUTH_USER_MODEL = 'users.UserProfile'

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware', # Must be first
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

# Custom user model
AUTH_USER_MODEL = 'users.UserProfile'

# Internationalization
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'Asia/Kolkata'
USE_I18N = True
USE_TZ = True

# Default primary key field
DEFAULT_AUTO_FIELD = 'django.db.models.UUIDField'

# config/settings/development.py
from .base import *

DEBUG = True
ALLOWED_HOSTS = ['*']

# Use local PostgreSQL

```

```

DATABASES = {
    'default': {
        'ENGINE': 'django.contrib.gis.db.backends.postgis',
        'NAME': env('DB_NAME'),
        'USER': env('DB_USER'),
        'PASSWORD': env('DB_PASSWORD'),
        'HOST': env('DB_HOST', default='localhost'),
        'PORT': env('DB_PORT', default='5432'),
    }
}

# Use local Redis
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {'hosts': [env('REDIS_URL')]},
    }
}

# Use local file system for media in dev (not S3)
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

# Django debug toolbar
INSTALLED_APPS += ['debug_toolbar']
MIDDLEWARE = ['debug_toolbar.middleware.DebugToolbarMiddleware']
            + MIDDLEWARE
INTERNAL_IPS = ['127.0.0.1']

# Email backend (console in dev – see all emails in terminal)
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'

# Logging
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'handlers': {'console': {'class': 'logging.StreamHandler'}},
    'root': {'handlers': ['console'], 'level': 'DEBUG'},
    'loggers': {
        'django.db.backends': {'level': 'DEBUG', 'handlers':
            ['console']},
    },
}

# config/settings/production.py
from .base import *
import sentry_sdk
from sentry_sdk.integrations.django import DjangoIntegration
from sentry_sdk.integrations.celery import CeleryIntegration

DEBUG = False
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS')

```

Security headers

```
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_BROWSER_XSS_FILTER = True
X_FRAME_OPTIONS = 'DENY'
```

PostgreSQL with SSL

```
DATABASES = {
    'default': {
        'ENGINE': 'django.contrib.gis.db.backends.postgis',
        'NAME': env('DB_NAME'),
        'USER': env('DB_USER'),
        'PASSWORD': env('DB_PASSWORD'),
        'HOST': env('DB_HOST'),
        'PORT': '5432',
        'CONN_MAX_AGE': 60,
        'OPTIONS': {'sslmode': 'require'},
    }
}
```

Redis Channel Layer

```
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
            'hosts': [env('REDIS_URL')],
            'capacity': 1500,
            'expiry': 10,
        },
    },
}
```

S3 Storage

```
DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage'
AWS_ACCESS_KEY_ID = env('AWS_ACCESS_KEY_ID')
AWS_SECRET_ACCESS_KEY = env('AWS_SECRET_ACCESS_KEY')
AWS_STORAGE_BUCKET_NAME = env('AWS_S3_BUCKET_NAME')
AWS_S3_REGION_NAME = env('AWS_S3_REGION_NAME')
AWS_S3_FILE_OVERWRITE = False
AWS_DEFAULT_ACL = 'private'
AWS_S3_SIGNATURE_VERSION = 's3v4'
```

Sentry

```
sentry_sdk.init(
    dsn=env('SENTRY_DSN'),
    integrations=[DjangoIntegration(), CeleryIntegration()],
    traces_sample_rate=0.2,
    send_default_pii=False,
)
```

7. URL Routing

```
# config/urls.py
from django.contrib import admin
from django.urls import path, include
from drf_spectacular.views import SpectacularAPIView,
    SpectacularSwaggerUIView

urlpatterns = [
    # Django admin (internal use only)
    path('django-admin/', admin.site.urls),

    # API Schema (dev only – disable in production or put behind
    # auth)
    path('api/schema/', SpectacularAPIView.as_view(),
        name='schema'),
    path('api/docs/',
        SpectacularSwaggerUIView.as_view(url_name='schema'),
        name='swagger-ui'),

    # Health check
    path('api/health/', include('apps.core.urls')),

    # Authentication
    path('api/auth/', include('apps.authentication.urls')),

    # User-facing APIs
    path('api/users/', include('apps.users.urls')),
    path('api/guards/', include('apps.guards.urls')),
    path('api/bookings/', include('apps.bookings.urls')),
    path('api/tracking/', include('apps.tracking.urls')),
    path('api/payments/', include('apps.payments.urls')),
    path('api/notifications/',
        include('apps.notifications.urls')),
    path('api/sos/', include('apps.sos.urls')),
    path('api/reviews/', include('apps.reviews.urls')),

    # Admin Panel APIs (protected by IsAdminUser permission)
    path('api/admin/', include('apps.admin_panel.urls')),
]

# config/asgi.py
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.security.websocket import import
    AllowedHostsOriginValidator

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
    'config.settings.production')
```

```

django_asgi_app = get_asgi_application()

from utils.ws_middleware import JWTAuthMiddlewareStack
from apps.tracking.routing import tracking_websocket_urlpatterns
from apps.sos.routing import sos_websocket_urlpatterns
from apps.admin_panel.routing import admin_websocket_urlpatterns

application = ProtocolTypeRouter({
    # HTTP requests → regular Django views
    'http': django_asgi_app,

    # WebSocket requests → Channels consumers
    'websocket': AllowedHostsOriginValidator(
        JWTAuthMiddlewareStack(
            URLRouter(
                tracking_websocket_urlpatterns +
                sos_websocket_urlpatterns +
                admin_websocket_urlpatterns
            )
        )
    ),
})

```

8. ASGI vs WSGI

b-secure uses **ASGI** (not WSGI) to support WebSocket connections alongside regular HTTP.

WSGI (traditional Django):

HTTP request → Django → HTTP response

Cannot hold persistent connections → no WebSocket support

ASGI (Django Channels):

HTTP request → Django views (same as WSGI)

WS connect → Channels consumer (persistent connection)

Both run in the same process under Daphne / Uvicorn

Server: Daphne (official Channels ASGI server)

Production command

```

daphne -b 0.0.0.0 -p 8000 \
    --access-log /var/log/daphne/access.log \
    --proxy-headers \
    config.asgi:application

```

Workers: 4 Daphne processes behind Nginx (each handling both HTTP and WS).

9. Code Style & Conventions

Python Style

- **PEP 8** compliance enforced via `flake8`
- **Black** for auto-formatting (line length: 88)
- **isort** for import ordering
- **Type hints** on all service methods and utility functions

```
# Format code
```

```
black .
```

```
isort .
```

```
# Lint
```

```
flake8 .
```

Pre-commit Hooks

```
# .pre-commit-config.yaml
```

```
repos:
```

```
- repo: https://github.com/psf/black  
  rev: 24.4.2
```

```
  hooks:
```

```
    - id: black
```

```
- repo: https://github.com/PyCQA/isort  
  rev: 5.13.2
```

```
  hooks:
```

```
    - id: isort
```

```
- repo: https://github.com/PyCQA/flake8  
  rev: 7.1.0
```

```
  hooks:
```

```
    - id: flake8
```

```
- repo: https://github.com/PyCQA/bandit  
  rev: 1.7.9
```

```
  hooks:
```

```
    - id: bandit
```

```
      args: ["-r", "apps/", "-ll"]
```

Naming Conventions

Item	Convention	Example
Models	PascalCase	GuardProfile, BookingSession
Model fields	snake_case	created_at, guard_type
API endpoints	kebab-case	/api/booking-requests/
Serializers		BookingCreateSerializer

Item	Convention	Example
	PascalCase + Serializer suffix	
ViewSets	PascalCase + ViewSet suffix	BookingViewSet
Services	PascalCase + Service suffix	BookingService
Celery tasks	snake_case with verb prefix	process_session_payment
Constants	UPPER_SNAKE_CASE	BOOKING_STATUS_ACTIVE

API Response Format

All API responses follow a consistent envelope format:

// Success (single object)

```
{
  "data": { ... },
  "message": "Booking created successfully"
}
```

// Success (list)

```
{
  "data": [ ... ],
  "count": 100,
  "next": "https://api.bsecure.in/api/bookings/?page=2",
  "previous": null
}
```

// Error

```
{
  "error": {
    "code": "BOOKING_NOT_FOUND",
    "message": "No booking found with the given ID",
    "details": {}
  }
}
```

// Validation Error

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": {
      "phone_number": ["Enter a valid phone number."],
      "service_type": ["This field is required."]
    }
  }
}
```

```
}  
}
```

10. Common Utilities

utils/permissions.py

```
from rest_framework.permissions import BasePermission  
  
class IsGuard(BasePermission):  
    """Allows access only to verified guards."""  
    def has_permission(self, request, view):  
        return (  
            request.user.is_authenticated and  
            hasattr(request.user, 'guard_profile') and  
            request.user.guard_profile.verification_status ==  
            'ACTIVE'  
        )  
  
class IsVerifiedUser(BasePermission):  
    """Allows access only to active (non-suspended) users."""  
    def has_permission(self, request, view):  
        return (  
            request.user.is_authenticated and  
            not request.user.is_suspended  
        )  
  
class IsBookingParticipant(BasePermission):  
    """Allows access only if the user is the booking's user or  
    guard."""  
    def has_object_permission(self, request, view, obj):  
        return (  
            obj.user == request.user or  
            (hasattr(request.user, 'guard_profile') and  
             obj.guard == request.user.guard_profile)  
        )  
  
class IsAdminUser(BasePermission):  
    """Restricts access to platform staff/admin only."""  
    def has_permission(self, request, view):  
        return request.user.is_authenticated and  
            request.user.is_staff
```

utils/helpers.py

```
import random  
import string  
import hashlib  
from django.utils import timezone
```

```

def generate_otp(length=4) -> str:
    """Generate a numeric OTP of given length."""
    return ''.join(random.choices(string.digits, k=length))

def hash_otp(otp: str) -> str:
    """One-way hash an OTP for safe storage."""
    return hashlib.sha256(otp.encode()).hexdigest()

def verify_otp(raw_otp: str, hashed_otp: str) -> bool:
    """Verify a raw OTP against its stored hash."""
    return hash_otp(raw_otp) == hashed_otp

def is_within_radius(lat1, lng1, lat2, lng2, radius_km: float) -> bool:
    """Quick Haversine check – use PostGIS for production queries."""
    from math import radians, cos, sin, asin, sqrt
    R = 6371 # Earth radius in km
    dlat = radians(lat2 - lat1)
    dlng = radians(lng2 - lng1)
    a = sin(dlat/2)**2 + cos(radians(lat1)) * cos(radians(lat2))
        * sin(dlng/2)**2
    return 2 * R * asin(sqrt(a)) <= radius_km

def mask_phone_number(phone: str) -> str:
    """Return masked phone for display: +91*****7890"""
    if len(phone) < 4:
        return phone
    return phone[:3] + '*' * (len(phone) - 6) + phone[-4:]

```

utils/storage.py

```

import boto3
from django.conf import settings

def generate_presigned_url(s3_key: str, expiry_seconds: int = 900) -> str:
    """
    Generate a pre-signed URL for private S3 objects.
    Default expiry: 15 minutes.
    """
    s3_client = boto3.client(
        's3',
        region_name=settings.AWS_S3_REGION_NAME,
        aws_access_key_id=settings.AWS_ACCESS_KEY_ID,
        aws_secret_access_key=settings.AWS_SECRET_ACCESS_KEY,

```

```

)
return s3_client.generate_presigned_url(
    'get_object',
    Params={'Bucket': settings.AWS_STORAGE_BUCKET_NAME,
            'Key': s3_key},
    ExpiresIn=expiry_seconds,
)

```

utils/exceptions.py

```

from rest_framework.views import exception_handler
from rest_framework.response import Response
from rest_framework import status

def custom_exception_handler(exc, context):
    """Wrap all DRF errors in the standard b-secure error
    envelope."""
    response = exception_handler(exc, context)

    if response is not None:
        error_data = {
            'error': {
                'code': _get_error_code(response.status_code),
                'message': _flatten_errors(response.data),
                'details': response.data if
                    isinstance(response.data, dict) else {},
            }
        }
        response.data = error_data

    return response

def _get_error_code(status_code: int) -> str:
    codes = {
        400: 'VALIDATION_ERROR',
        401: 'AUTHENTICATION_REQUIRED',
        403: 'PERMISSION_DENIED',
        404: 'NOT_FOUND',
        429: 'RATE_LIMIT_EXCEEDED',
        500: 'INTERNAL_SERVER_ERROR',
    }
    return codes.get(status_code, 'ERROR')

def _flatten_errors(data) -> str:
    if isinstance(data, dict):
        for key, value in data.items():
            if isinstance(value, list) and value:
                return str(value[0])
    elif isinstance(data, list) and data:

```

```
    return str(data[0])  
return str(data)
```